

Practical -1 _OS_2520090235

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];

    // Accept linux command from the user
    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    // Create child process
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    // Child Process
    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("\nExecuting command: %s\n\n", command);

        // Executes linux command
        execlp(command, command, NULL);

        // Executes only if exec fails
        printf("Command execution failed!\n");
        exit(1);
    }

    // Parent Process
    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        // wait for child process
        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

OUTPUT:

```

rohita@LaxmiDurga: $ ./a.out
Enter a Linux command: ls

Parent Process
Parent PID : 1240
Child PID  : 1241

Child Process
Child PID  : 1241
Parent PID : 1240

Executing command: ls

First.c Hello-World Ossp Practical2.c Skill2.c Skill3.c Skill3Task2.c a.out command_exec.c nano.565.save ossp skil4Task1.c skill3Task2.c

Child process completed.

```

Observation:

- The program successfully creates a child process using `fork()` and executes the entered Linux command using `exec()`.
- The parent process waits for the child using `wait()` and displays the PIDs of both parent and child processes.

TASK -2 : Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Uname -a : Identifies the core system kernel and hardware architecture.

```

Parent Process
Parent PID : 1240
Child PID  : 1241

Child Process
Child PID  : 1241
Parent PID : 1240

Executing command: ls

First.c Hello-World Ossp Practical2.c Skill2.c Skill3.c Skill3Task2.c a.out command_exec.c nano.565.save ossp skil4Task1.c skill3Task2.c

Child process completed.
rohita@LaxmiDurga: $ nano Practical2.c
rohita@LaxmiDurga: $ uname -a
Linux LaxmiDurga 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
rohita@LaxmiDurga: $ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:   0-3
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) i3-5010U CPU @ 2.10GHz
CPU family:             6
Model:                  61
Thread(s) per core:     2
Core(s) per socket:     2
Socket(s):              1
Stepping:               4
BogoMIPS:               4190.30
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm arch
2 movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch pti ssbd ibrs ibpb stibp fsgsbase bmi1 avx2 smep bmi2 erms
Virtualization features:
Hypervisor vendor:      Microsoft
Virtualization type:    full
Caches (sum of all):
L1d:                     64 KiB (2 instances)
L1i:                     64 KiB (2 instances)
L2:                      512 KiB (2 instances)
L3:                      3 MiB (1 instance)
NUMA:
NUMA node(s):            1
NUMA node0 CPU(s):      0-3
Vulnerabilities:
Gather data sampling:    Not affected
Ghostwrite:              Not affected
Indirect target selection: Mitigation; Aligned branch/return thunks

```

Lscpu : Details the physical CPU geometry (sockets, cores, hardware caches).

```

rohita@LaxmiDurga:~$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.2  0.3 24376 15388 ?        Ss   16:08   0:03 /usr/lib/systemd/systemd --system --deserialize=79
root           2  0.0  0.0   3180  2208 hvc0     Sl+  16:08   0:00 /init
root           8  0.0  0.0   3180  2048 hvc0     Sl+  16:08   0:00 plan9 --control-socket 8 --log-level 4 --server-fd 9 --pipe-fd 11 --log-truncate
root          121  0.0  0.0   3184  1112 ?        Ss   16:08   0:00 /init
root          122  0.0  0.0   3200  1252 ?        S    16:08   0:00 /init
rohita        123  0.0  0.1  6268  5544 pts/0    Ss   16:08   0:00 bash
root          355  0.0  0.0   2888  1944 ?        Ss   16:09   0:00 /bin/sh /usr/lib/systemd/scripts/chronyd-starter.sh -n -F 1
root          356  0.0  0.0   4472  3080 ?        Ss   16:09   0:00 /usr/sbin/cron -f -P
message+     358  0.0  0.1  9012  5652 ?        Ss   16:09   0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root          365  0.0  0.7 44096 29840 ?        Ss   16:09   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root          370  0.0  0.2 17788  8792 ?        Ss   16:09   0:00 /usr/lib/systemd/systemd-logind
root          390  0.0  0.0   5492  2756 tty1     Sst+ 16:09   0:00 /usr/sbin/agetty --noreset --noclear --issue-file=/etc/issue:/etc/issue.d:/run/issue.d:/usr/lib/issue.d - linux
syslog       412  0.0  0.1 220548  5664 ?        Ssl  16:09   0:00 /usr/sbin/rsyslogd -n -iNONE
root          415  0.0  0.8 123460 32644 ?        Ssl  16:09   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
_chrony      434  0.0  0.2  24036 11492 ?        S    16:09   0:00 /usr/sbin/chronyd -n -F 1 -x
_chrony      464  0.0  0.0  12096  2340 ?        S    16:10   0:00 /usr/sbin/chronyd -n -F 1 -x
root          916  0.0  0.3 34088 12668 ?        S<s  16:14   0:00 /usr/lib/systemd/systemd-journald
root          976  0.0  0.2 33776 10792 ?        Ss   16:14   0:00 /usr/lib/systemd/systemd-udev
systemd+    1037  0.0  0.3  22416 14472 ?        Ss   16:15   0:00 /usr/lib/systemd/systemd-resolved
polkitd     1153  0.0  0.2 306444  8636 ?        Ssl  16:17   0:00 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
rohita     1284  0.0  0.1   7160  4428 pts/0    R+   16:32   0:00 ps aux

```

Ps aux: Gives a idea of running software processes and their resource states

Top: Shows a real-time, dynamic view of active CPU and Virtual Memory/Swap consumption.

```

top - 16:34:20 up 26 min,  0 users,  load average: 0.00, 0.05, 0.26
Tasks: 21 total,  1 running, 20 sleeping,  0 stopped,  0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  3876.6 total,  3350.4 free,   486.7 used,   181.3 buff/cache
MiB Swap:  1024.0 total,  1024.0 free,    0.0 used.  3389.9 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0  24376  15388 11324  S   0.0   0.4   0:03.35 systemd
    2 root        20   0   3180   2208  2072  S   0.0   0.1   0:00.16 init-systemd(Ub
    8 root        20   0   3180   2048  1940  S   0.0   0.1   0:00.00 init
   121 root        20   0   3184   1112   980  S   0.0   0.0   0:00.00 SessionLeader
   122 root        20   0   3200   1252  1108  S   0.0   0.0   0:00.02 Relay(123)
   123 rohita      20   0   6268   5544  3864  S   0.0   0.1   0:00.16 bash
   355 root        20   0   2888   1944  1820  S   0.0   0.0   0:00.15 chronyd-starter
   356 root        20   0   4472   3080  2832  S   0.0   0.1   0:00.02 cron
   358 message+    20   0   9012   5652  4684  S   0.0   0.1   0:00.49 dbus-daemon
   365 root        20   0 44096 29840 14696  S   0.0   0.8   0:00.30 networkd-dispat
   370 root        20   0 17788   8792  7580  S   0.0   0.2   0:00.27 systemd-logind
   390 root        20   0   5492   2756  2572  S   0.0   0.1   0:00.01 agetty
   412 syslog      20   0 220548   5664  4364  S   0.0   0.1   0:00.08 rsyslogd
   415 root        20   0 123460 32644 18036  S   0.0   0.8   0:00.23 unattended-upgr
   434 _chrony      20   0  24036 11492  7132  S   0.0   0.3   0:00.17 chronyd
   464 _chrony      20   0  12096  2340  1696  S   0.0   0.1   0:00.00 chronyd
   916 root        19  -1 34088 12668 11492  S   0.0   0.3   0:00.06 systemd-journal
   976 root        20   0 33776 10792  9108  S   0.0   0.3   0:00.10 systemd-udev
  1037 systemd+    20   0  22416 14472 11980  S   0.0   0.4   0:00.10 systemd-resolve
  1153 polkitd      20   0 306444   8636  7820  S   0.0   0.2   0:00.15 polkitd
  1294 rohita      20   0   8276   6036  3908  R   0.0   0.2   0:00.00 top

```

Observation:

- The commands show how Linux monitors and manages CPU, memory, storage, and processes.
- The OS abstracts hardware resources and provides them efficiently to applications.